Learning Dense Log-Pile Clearing from Vision in a Grasp-and-Remove Crane Simulator

Anonymous Authors

***Abstract*—Clearing dense log piles is a contact-rich manipulation problem with a large, evolving state space and a difficult exploration landscape. We study grasp target selection for a forestry log loader in simulation, focusing on where and how to orient a grapple to clear dense piles of 200 rigid-body logs in frictional contact. We introduce a grasp-and-remove formulation in which each environment step is a complete pick attempt: a policy selects a grasp target (x, y, z, ψ), a fixed finite-state machine (FSM) executes the approach and lift, and grasped logs are removed. Policies observe raw depth-derived point clouds—requiring no semantic segmentation—processed by a PointNet encoder. We compare four methods: a privileged-state heuristic, from-scratch reinforcement learning (RL) with PPO, behavior cloning (BC) from heuristic demonstrations, and BC-initialized RL fine-tuning (BC+RL) with an asymmetric actor-critic. From-scratch RL stalls at roughly 60% pile clearing regardless of observation type. BC achieves 98.6% clearing from raw point clouds alone, matching the 97.9% heuristic expert. BC+RL combines BC's full-trajectory coverage with RL's per-grasp optimization, achieving the highest grasp success rate (87.0%) and grapple stability (0.950) while maintaining 98.3% clearing. Our results show that raw point clouds are sufficient for sequential grasp targeting and that demonstration data provides the exploration curriculum that from-scratch RL cannot discover.**

***Index Terms*—robot learning, multi-object grasping from dense clutter, behavior cloning, reinforcement learning fine-tuning, contact-rich simulation**

I. Introduction

Forestry operations increasingly seek autonomy for heavy machinery to improve safety and productivity [**?**]. A key subtask is log-pile clearing: repeatedly grasping multiple logs from a rack or pile and removing them. This setting is challenging because (i) the scene is densely cluttered with hundreds of rigid-body logs in frictional contact, (ii) each grasp disturbs the pile through contact cascades, continuously reshaping the configuration for subsequent attempts, and (iii) no two pile configurations are alike, so a targeting strategy must be robust to diverse arrangements.

We address the grasp target selection problem: given a raw point cloud derived from a stereo depth camera—without semantic segmentation—where should the crane place and orient the grapple to clear the pile efficiently? We deliberately separate this high-level targeting decision from low-level motion execution. Concretely, we implement a grasp-and-remove simulator in Isaac Lab [**?**] where each environment step is one complete pick attempt: a policy selects where to grasp, a fixed FSM handles execution, and grasped logs are removed rather than deposited, allowing us to measure clearing progress without modeling placement dynamics.

Existing work on autonomous log grasping addresses small-scale settings: single-log pick-and-place, small groupings of 2–7 logs, or grasp prediction without sequential clearing. We scale to full pile-level clearing with dense frictional contact dynamics among hundreds of logs, where each grasp reshapes the configuration for all subsequent attempts. To our knowledge, this is the first study of learned sequential grasp targeting at this pile scale.

Our main contributions are:

- A grasp-and-remove crane environment in Isaac Lab for dense pile clearing at scale (200 logs), with a fixed FSM that isolates targeting from execution and supports GPU-parallelized training.
- A BC→RL fine-tuning pipeline using PointNet on raw (unsegmented) point clouds with a frozen encoder and asymmetric actor-critic [**?**], achieving near-expert clearing from depth alone.
- A four-method comparison showing: (a) from-scratch RL fails regardless of observation type, (b) BC achieves near-expert clearing from raw point clouds, and (c) BC+RL improves per-grasp success rate and stability while maintaining clearing performance.

II. Related Work

A. Grasping in Clutter

Learning-based grasp detection has progressed from single isolated objects to cluttered, multi-object scenes, with applications spanning industrial bin picking of manufactured parts [**?**], warehouse depalletizing, and natural-resource handling such as forestry log loading. Deep grasp prediction networks [**?**], [**?**] predict grasp pose directly from visual input. Sequential decluttering has been studied in bin picking [**?**], [**?**], including learning grasp sequences that account for how each pick changes the scene [**?**]. However, these works typically involve heterogeneous objects of varied shape and size in loose arrangements, where clutter arises from random placement rather than dense packing. The grasping primitive is almost always a precision or pinch grasp that isolates a single target object. Our setting differs in important ways: we perform power grasps from dense piles of cylindrical logs of uniform geometry that are initially stacked in ordered layers. The homogeneity and tight packing mean that logs are in extensive frictional contact with their neighbors, and each grasp attempt (which captures multiple logs simultaneously) disturbs the pile, causing settling, rolling, and potential knock-offs that reshape the configuration for all subsequent attempts. For

3D perception, PointNet [**?**] processes unordered point sets via per-point MLPs and symmetric pooling; we adopt this architecture to encode depth-derived point clouds into a latent representation for grasp target prediction.

B. Learning for Log Grasping and Forestry Crane Control

A growing body of work applies learning methods to log grasping and forestry crane automation, though existing approaches address smaller-scale settings than the one considered here.

For grasp prediction, Wallin et al. [**?**] use RL with virtual visual servoing for multi-log grasping from small groupings of 2–5 logs in simulation, separating image segmentation from Cartesian crane control; their agent achieves 95% success but the pile dynamics are limited to a handful of objects. Fälldin et al. [**?**] train a U-Net CNN on synthetic RGB-D images to predict grasp poses for small clusters of up to 7 logs, with an open-loop kinematic controller; their work focuses on grasp prediction from synthetic data and does not address sequential clearing or pile-scale dynamics. Ayoub et al. [**?**] propose a CNN-based grasp planner for a log-loading forestry machine that predicts a grasp configuration from a depth image for small piles, validated both in simulation and on a commercial log loader retrofitted for autonomy.

For crane control, Vu et al. [**?**] present a MuJoCo-based forestry crane simulator and benchmark RL velocity controllers for grasping single logs of varying diameter (simulation only). Kurinov et al. [**?**] train an RL agent for the full single-log pick-and-place pipeline (locate, grasp, transport, load) in Isaac Gym simulation, achieving 94% success across parallel environments but with one log at a time. Jebellat et al. [**?**] use dynamic programming to design anti-sway trajectories for a log loader end effector, addressing the pendulum dynamics that also arise in our passive grapple chain. These works address individual subtasks but none tackle sequential clearing of a dense pile at the scale considered here.

More broadly, using demonstrations to initialize RL policies has been shown to overcome exploration challenges in contact-rich manipulation [**?**], [**?**], and asymmetric actor-critic architectures—where the critic receives privileged state information unavailable to the actor—have proven effective for learning from high-dimensional visual observations [**?**]. To our knowledge, no prior work combines these ideas for sequential pile clearing from point clouds.

III. Task Formulation

A. Grasp-and-Remove Formulation

Each environment step corresponds to a complete grasp attempt: a policy selects a grasp target, the FSM executes the approach, close, and lift, and grasped logs are removed from the scene. This formulation is shared by all methods (heuristic, BC, RL, and BC+RL). The state $s_t \in \mathcal{S}$ is the current pile configuration (observed via point cloud or privileged poses; Section **??**). A grasp target is defined by the tuple

$$\mathbf{g} = (x,\, y,\, z,\, \psi), \tag{1}$$

where (x, y, z) is the grapple position in the crane base frame and ψ is the grapple yaw angle about the vertical axis. Because a cylindrical log at yaw ψ is identical to the same log at $\psi + \pi$, the policy represents ψ via the π-periodic encoding $(\cos 2\psi, \sin 2\psi)$ [**?**], giving a 5D action $a_t \in \mathcal{A} \subset \mathbb{R}^5$ (Section **??**). The transition $T(s_{t+1}|s_t, a_t)$ is determined by the physics simulation and FSM execution. After each grasp attempt, an outcome evaluation measures the number of logs grasped, grapple–log alignment, and grapple stability (Section **??**). For RL training, this outcome is used as a scalar reward $r_t = R(s_t, a_t)$, and the full structure is treated as an episodic Markov decision process $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$ with discount factor γ, where a policy π maximizes the expected discounted return over a horizon of H grasp attempts:

$$\max_{\pi}\ \mathbb{E}_{\pi}\left[\sum_{t=0}^{H-1} \gamma^t\, r_t\right]. \tag{2}$$

An episode terminates when no logs remain on the rack or after a fixed horizon of $H = 30$ grasp attempts. A log leaves the rack either by being grasped and removed or by being knocked outside the workspace bounds (Section **??**); both reduce the remaining count for termination purposes, but only intentionally grasped logs are credited toward the clearing rate metric (Section **??**). The horizon is chosen to exceed the minimum needed: the grapple can hold up to 20 logs per grasp, and the heuristic baseline clears 200-log piles in approximately 20 attempts, so $H = 30$ provides a 50% margin while keeping episodes tractable.

B. Observations

The policy observes a raw (unsegmented) point cloud of the scene. A depth camera mounted on the crane renders the scene from above the rack, producing a depth image D_t. The simulated camera uses a pinhole model with parameters matching the Stereolabs ZED X in wide-angle configuration[1] (focal length 2.2 mm, sensor width 5.8 mm, 640×360 resolution, depth range 0.3–20 m).

The observation is constructed in three steps. First, each pixel (u, v) with a valid depth reading within the sensor range is back-projected to a 3D point using the camera intrinsic matrix $\mathbf{K} \in \mathbb{R}^{3\times3}$:

$$\mathcal{P}_w = \left\{ \mathbf{K}^{-1}(u, v, 1)^{\top} \cdot D_t(u, v) \;\middle|\; d_{\min} \le D_t(u, v) \le d_{\max} \right\}, \tag{3}$$

where the camera-frame points are transformed to the world frame using the known camera extrinsics. Second, each point $\mathbf{p} \in \mathcal{P}_w$ is transformed into the crane base frame using the base-frame rotation $\mathbf{R}_b$ and translation $\mathbf{t}_b$:

$$\mathcal{P}_b = \left\{ \mathbf{R}_b^{\top}(\mathbf{p} - \mathbf{t}_b) \;\middle|\; \mathbf{p} \in \mathcal{P}_w \right\}. \tag{4}$$

Third, Farthest Point Sampling (FPS) downsamples $\mathcal{P}_b$ to a fixed-size set of $N_p = 1024$ points, yielding the observation:

$$o_t = \mathrm{FPS}(\mathcal{P}_b,\ N_p)\ \in \mathbb{R}^{N_p \times 3}. \tag{5}$$

[1] https://www.stereolabs.com/products/zed-x

If the cloud contains fewer than N_p points (e.g., near the end of an episode), the remaining rows are zero-padded. Crucially, no semantic segmentation is applied: the raw point cloud includes all surfaces visible to the depth camera (logs, rack, ground). The PointNet encoder must implicitly learn to attend to task-relevant geometry. In our experiments, we find that raw point clouds match or exceed the performance of segmented point clouds in which a semantic mask isolates log pixels before back-projection (Section **??**). This result has practical significance for sim-to-real transfer, as it removes the dependency on a trained segmentation model [**?**].

As an ablation, we also evaluate policies that observe privileged per-object state: the top-K log poses sorted by height (highest first), each represented as (x, y, z, ψ) in the crane base frame. Removed logs are excluded and remaining slots are padded with -1. With $K = 32$ this yields a compact $4K = 128$ dimensional vector. This baseline isolates the effect of the perception gap: any performance difference between pose and point cloud policies is attributable to the observation representation rather than the learning algorithm.

C. Actions and Workspace Scaling

The policy outputs a 5D vector $\tilde{a}_t \in \mathbb{R}^5$ (before squashing). The first three components encode position and the remaining two encode grapple yaw via a cosine–sine pair.

Each position component is squashed via $\tanh$ and then linearly mapped to the per-environment workspace bounds to produce the target coordinate p_i:

$$p_i = \frac{(\tanh \tilde{a}_i + 1)}{2}(b_i^{\max} - b_i^{\min}) + b_i^{\min}, \quad i \in \{x, y, z\}, \tag{6}$$

where $b_i^{\min}, b_i^{\max}$ are the rack-aligned workspace limits for each axis. These bounds are computed from the rack geometry with margins of 1.0 m along x (front and back), 1.15 m along y, and extend from 0.05 m below the rack base to 0.35 m above the top of the pile in z. The resulting workspace is approximately $2.0 \times 5.2 \times 1.0$ m in the crane base frame. The bounds serve a dual purpose. First, they define the policy's reachable target space, ensuring that all output actions map to physically executable grasp poses. Second, logs that drift outside these bounds during a grasp attempt (e.g., knocked off the rack by grapple contact) are automatically removed before the next targeting decision and counted separately (see Section **??**), preventing unreachable logs from accumulating. The heuristic expert (Section **??**), which bypasses the policy's action scaling and targets log poses directly, operates within the same spatial bounds.

The remaining two components encode the grapple yaw angle ψ about the vertical axis. A cylindrical log at yaw ψ is physically identical to the same log at $\psi + \pi$. A scalar representation forces the network to learn this symmetry from data; instead, following the grasp-angle encoding of Morrison et al. [**?**], we encode ψ as the pair $(\cos 2\psi,\ \sin 2\psi)$, which has period π and thus maps symmetric orientations to the same point. The policy outputs two scalars $(\tilde{a}_4, \tilde{a}_5)$; after squashing, the yaw is recovered as

$$\psi = \tfrac{1}{2} \operatorname{atan2}\big(\tanh \tilde{a}_5,\ \tanh \tilde{a}_4\big) \in \big[-\tfrac{\pi}{2},\ \tfrac{\pi}{2}\big]. \tag{7}$$

In our imitation pipeline, expert targets are encoded in the same space: positions are normalized and passed through $\operatorname{arctanh}$, and yaw is stored directly as $(\cos 2\psi, \sin 2\psi)$. This ensures BC and RL policies operate in an identical action space.

IV. Environment and Controller

A. Low-level Execution FSM

All methods share the same fixed FSM controller, which runs at the control rate (60 Hz, i.e., 120 Hz physics with $2\times$ decimation) inside each environment step. Given a grasp target (x, y, z, ψ), produced by the heuristic (Section **??**) or a learned policy (BC or RL), the FSM executes a complete grasp cycle.

The crane arm has four actuated joints (basemast, boom, stick, and telescope) followed by a two-link passive chain that suspends the grapple from the telescope tip. The passive joints (`upperpassive` and `lowerpassive`) are unactuated revolute joints with low stiffness and damping, modeling the gravity-driven hanging behavior of a real forestry crane's grapple attachment. Below the passive chain, a yaw joint (`lowerpassive_to_basegrapple`) actively rotates the grapple about the vertical axis, and two gripper joints control the jaw opening.

In this kinematic structure, the IK controller commands the `upperpassive` link (the top of the passive chain), but the grapple hangs $\sim$0.4 m below it and swings like a pendulum during arm movements. Consequently, all grasp targets are specified for the `basegrapple` body and then offset vertically to the `upperpassive` IK target. The FSM uses a two-stage convergence check at each phase: it first waits for the `upperpassive` link to reach its target within $\epsilon_{\text{up}} = 0.10$ m, then waits for the `basegrapple` body to settle within $\epsilon_{\text{bg}} = 0.25$ m. Both must hold for $N_{\text{dwell}} = 30$ consecutive timesteps before the phase advances, ensuring that pendulum oscillations have damped before proceeding.

The FSM implements a five-phase grasp cycle:

1) `HOVER`: Move grapple to hover position above the target $(x, y, z + z_{\text{clear}})$, with $z_{\text{clear}} = 2.5$ m clearance above the target log.
2) `ALIGN`: Hold position and rotate grapple to the target yaw ψ.
3) `DESCEND`: Lower grapple to approach height at reduced speed (30% step scaling) to minimize pile disturbance.
4) `CLOSE`: Close the grapple jaws incrementally via discrete stepping.
5) `LIFT`: Lift back to hover height at moderate speed (50% step scaling), then hold for $\sim$1 s while grasp outcome is evaluated (Section **??**). Grasped logs are removed, the jaws open, and the cycle repeats from `HOVER` with the next target.

A per-phase timeout (100 timesteps, $\sim$2 s) forces advancement if the two-stage convergence is not reached, preventing the FSM from stalling on difficult configurations.

The arm is controlled using Isaac Lab's task-space Differential IK controller [**?**] (damped least-squares, position command type), which computes joint position targets from the current end-effector pose and Jacobian. The grapple yaw joint is controlled independently via a PD position controller. The grapple jaws are controlled using a PD velocity law with saturation and discrete stepping to avoid abrupt contact forces.

B. Outcome Evaluation

After lifting (`LIFT`), the FSM holds the grapple at hover height and evaluates grasp outcome by measuring proximity and orientation of logs relative to the `basegrapple` body:
Logs grasped (g): the number of active logs whose centers lie within a proximity radius of the grapple (default 1.5 m).

Alignment (α): for each grasped log i, we measure the angle between the grapple's jaw axis $\hat{y}_g$ and the log's length axis $\hat{y}_{\ell_i}$ via their dot product ($\hat{y}_g^\top \hat{y}_{\ell_i} = \cos\theta_a$, where θ_a is the misalignment angle). Because a cylindrical log has no preferred length direction, the even exponent absorbs the sign; raising to the 8th power creates a sharp dropoff that penalizes moderate misalignment:

$$\alpha_i = \left(\hat{y}_g^\top \hat{y}_{\ell_i}\right)^8, \qquad \alpha = \frac{1}{g}\sum_{i=1}^{g} \alpha_i \ \in [0,1]. \tag{8}$$

Stability (s): the grapple's levelness after lifting, measured as $\hat{z}_g^\top \hat{z}_w = \cos\theta_s$, where θ_s is the tilt angle between the grapple's up vector and the world vertical. Clamping to zero excludes the degenerate case of an inverted grapple; raising to the 4th power ensures high scores only when the grapple is nearly level:

$$s = \left(\max(0,\ \hat{z}_g^\top \hat{z}_w)\right)^4 \ \in [0,1]. \tag{9}$$

Logs identified as grasped are marked inactive and teleported out of the scene. Logs that leave the workspace bounds during a grasp are removed and excluded from the grasp count g.

We implement the environment in Isaac Lab and run PhysX GPU dynamics for scalable data collection. Log-on-log static and kinetic friction coefficients are set to $\mu_s = 0.62$ and $\mu_k = 0.48$ respectively (oak on oak [**?**]), and log dimensions are based on measurements of representative forestry logs. We use conservative contact modeling (simplified collision geometry on the grapple, tuned contact and rest offsets, and damping limits for logs) to reduce interpenetration and numerical jitter while maintaining high throughput. Table **??** lists the full set of crane, log, and controller parameters.

V. Learning and Baselines

A. Heuristic Expert

We implement a hand-tuned heuristic expert that serves both as a competitive baseline and as the demonstration source for BC. The heuristic has access to privileged information (the position and orientation of every active log from the physics engine) and uses a greedy top-down strategy:

TODO: Insert zoomed-in grasp diagram showing: (1) the crane and grapple with the grasp state variables (x, y, z, ψ) labeled on the grapple body; (2) grapple holding logs with coordinate frames drawn on the grapple ($\hat{y}_g$, $\hat{z}_g$) and a log ($\hat{y}_\ell$), illustrating the alignment dot product $\langle \hat{y}_g, \hat{y}_\ell \rangle$ and the stability dot product $\hat{z}_g^\top \hat{z}_w$; (3) the 3D action space bounding box overlaid on the rack, showing the workspace limits $[b_i^{\min}, b_i^{\max}]$ that define the policy's reachable target space; (4) include outcome_scores.pdf subplots showing the shaped score curves ($(\cos\theta)^8$ for alignment, $(\cos\theta)^4$ for stability) alongside the diagram.

Fig. 1: Grasp geometry and action space. Left: coordinate frames used to compute alignment (α) and stability (s). The alignment score measures parallelism between the grapple jaw axis $\hat{y}_g$ and the log length axis $\hat{y}_\ell$; stability measures the grapple's levelness via $\hat{z}_g^\top \hat{z}_w$. Right: the 3D workspace bounds define the policy's action space over the rack.

At the start of each pick cycle (`HOVER` phase), the expert queries the simulator for the world-frame poses of all active logs in the environment. It selects the log with the highest z-coordinate (i.e., the topmost log in the pile), transforms its center position into the crane base frame, and sets this as the (x, y, z) grasp target. This greedy rule is effective because the highest log is typically the most accessible; it has the fewest obstructing neighbors and the grapple can reach it with minimal contact disturbance to the surrounding pile.

After positioning the grapple above the target, the expert computes an optimal yaw ψ that aligns the grapple's jaw axis with the target log's principal (length) axis. The target log's yaw is extracted from its world-frame quaternion and transformed into the crane base frame. Because a cylindrical log at yaw ψ is symmetric with the same log at $\psi + \pi$, the expert evaluates both orientations and selects the one requiring minimal joint rotation from the current configuration. This alignment is recomputed in the `ALIGN` phase using the grapple's live orientation, providing closed-loop yaw tracking rather than open-loop targeting.

The expert's greedy strategy is effective for dense piles: when many logs are stacked, the highest log is reliably graspable, and good alignment ensures that the grapple captures neighboring logs along the same layer. However, the heuristic makes no multi-step plan. In the endgame, when only a few scattered logs remain, targeting the highest log is no longer clearly optimal; logs may be isolated, partially wedged, or positioned where the grapple geometry makes alignment difficult. Despite this, the expert remains the strongest baseline in our experiments.

B. Behavior Cloning

We use BC [**?**] to train a policy from expert demonstrations. We collect demonstration pairs (o_t, a_t) from successful expert grasps only (failed grasps are discarded), where o_t is the point cloud observation and a_t is the expert action in the pre-

TABLE I: Simulation and controller parameters. Actuator gains are implicit PhysX joint-drive values; IK gains are the PD velocity controller used by the FSM.

Simulation	
Physics timestep (Δt)	$1/120$ s
Control decimation	$2\times$ (60 Hz)
Crane link masses	
Mast	290 kg
Main boom	172 kg
Stick	70 kg
Telescope	25 kg
Upperpassive	9 kg
Joint limits	
Basemast (revolute)	$\pm 100^\circ$
Boom (revolute)	-22° to $+75^\circ$
Stick (revolute)	-177° to $+2^\circ$
Telescope (prismatic)	0.13–1.80 m
Grapple (revolute)	$\pm 180^\circ$
Actuator gains (k_p / k_d)	
Main arm joints	10^6 / 10^5
Passive chain	300 / 500
Grapple yaw	5×10^4 / 2×10^4
Grapple tongs	0 / 500
IK controller PD gains (k_p / k_d)	
Arm joints	4.0 / 1.2
Grapple tongs	12.0 / 1.0
FSM parameters	
Hover clearance	2.5 m
Position tol. (upperpassive / basegrapple)	0.10 / 0.25 m
Dwell count	30 steps
Phase timeout	100 steps
Log properties	
Diameter / length	$\sim$0.3 / $\sim$3.0 m
Mass	15 kg
Static friction μ_s	0.62
Kinetic friction μ_k	0.48
Angular damping	0.05
Contact / rest offset	0.02 / 0.0 m
Max depenetration velocity	3.0 m/s

squashing space (positions via $\operatorname{arctanh}$, yaw via $\cos 2\psi, \sin 2\psi$ as described in Section **??**).

The policy uses a PointNet encoder [**?**]. A shared per-point MLP $h : \mathbb{R}^3 \to \mathbb{R}^{256}$ (layers $3 \to 64 \to 128 \to 256$ with batch normalization and ELU activations) maps each point independently; a symmetric max-pool aggregates the per-point features into a global latent vector:

$$\mathbf{z} = \max_{i=1,\dots,N_p} h\big(o_t^{(i)}\big) \ \in \mathbb{R}^{256}, \tag{10}$$

where $o_t^{(i)} \in \mathbb{R}^3$ is the i-th point in the observation. An actor MLP $\mu_\theta : \mathbb{R}^{256} \to \mathbb{R}^5$ (layers $256 \to 128 \to 64 \to 5$) maps the latent to the action $\tilde{a}_t = \mu_\theta(\mathbf{z})$. For the privileged-pose ablation, we replace the PointNet encoder with a three-layer MLP ($128 \to 256 \to 128 \to 64$) with ELU activations.

Table **??** summarizes the network architecture for both observation types.

TABLE II: Network architectures. All hidden layers use ELU activations. The PointNet encoder applies batch normalization. All policies output 5D actions: $(x, y, z, \cos 2\psi, \sin 2\psi)$.

	BC / RL (PCD)	**BC+RL (PCD)**	**RL (Pose)**
Encoder	PointNet: $3 \to 64 \to 128 \to 256$, BN, max-pool	Same (frozen, init from BC)	MLP: $128 \to 256 \to 128 \to 64$
Actor	$256 \to 128 \to 64 \to 5$	Same (init from BC)	Shared enc $\to 64 \to 5$
Critic	PointNet + MLP head	MLP: $128 \to 256 \to 128 \to 1$ (state)	MLP: $256 \to 128 \to 64 \to 1$

We apply dropout (rate 0.2) after the encoder. The BC objective minimizes the mean-squared error between the predicted and expert actions over a dataset $\mathcal{D}$ of successful grasps:

$$\mathcal{L}_{\text{BC}} = \frac{1}{|\mathcal{D}|} \sum_{(o,a^*)\in\mathcal{D}} \left\| \mu_\theta(\mathbf{z}) - a^* \right\|^2, \tag{11}$$

where $\mathbf{z}$ is the encoder output for observation o. We optimize with AdamW [**?**] (learning rate 10^{-4}, weight decay 10^{-4}) and ReduceLROnPlateau scheduling (factor 0.5, patience 10 epochs), with an 85/15 train/validation split and early stopping (patience 30 epochs). The training dataset contains 20,729 successful-grasp samples (17,620 train, 3,109 validation); the best model is selected by validation loss (0.051 MSE, epoch 246/250).

C. Reinforcement Learning

We train policies with PPO [**?**] using the RSL-RL library [**?**]. PPO optimizes a stochastic Gaussian policy with parameters θ:

$$\pi_\theta(a \mid s) = \mathcal{N}\big(\mu_\theta(s),\ \operatorname{diag}(\boldsymbol{\sigma}^2)\big), \tag{12}$$

where μ_θ outputs the mean action and $\boldsymbol{\sigma} \in \mathbb{R}^{|\mathcal{A}|}$ is a learnable standard deviation vector controlling exploration noise, initialized to σ_0 and updated by gradient descent alongside the actor weights. A separate critic network estimates the state-value function:

$$V_\phi(s) \approx \mathbb{E}_\pi\left[\sum_{t'=t}^{H-1} \gamma^{t'-t}\, r_{t'} \;\middle|\; s_t = s \right], \tag{13}$$

with parameters ϕ trained to minimize the prediction error against observed returns. Advantages are computed via Generalized Advantage Estimation (GAE) [**?**].

The actor uses the same PointNet encoder and actor MLP as the BC policy; the critic uses an independent PointNet encoder with its own MLP head. For the privileged-pose ablation, both actor and critic are separate MLPs with hidden dimensions $[256, 128, 64]$ and ELU activations. Key hyperparameters: $\sigma_0 = 1.0$, clipping ratio $\epsilon = 0.2$, entropy coefficient 0.01, learning rate 3×10^{-4} with adaptive KL-based scheduling (target KL 0.01), GAE with $\lambda = 0.95$, discount factor $\gamma = 0.99$, 4 steps per environment per iteration, 5 learning epochs with 4 mini-batches per epoch, and gradient clipping at norm 1.0. Each iteration collects $N_{\text{env}} \times 4$ grasp cycles (one

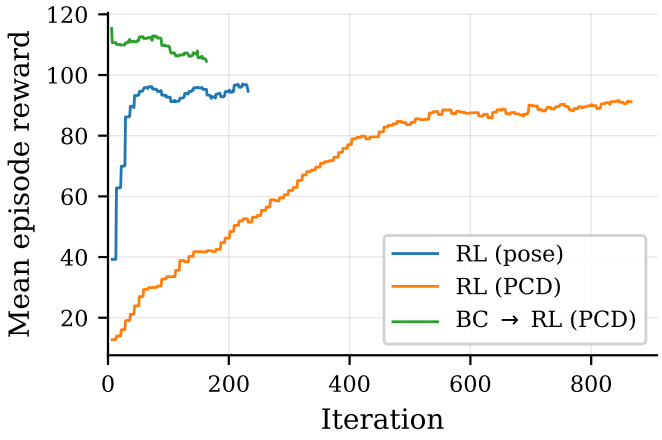

Fig. 2: PPO training curves. Mean episode reward (sum of per-grasp rewards $r = g \cdot \alpha \cdot s$ over a 200-log clearing episode) vs. PPO iteration. Each iteration collects $4N_{\text{env}}$ grasp cycles; flat segments occur when no episode has terminated since the last update. All three from-scratch RL variants (Pose, Seg. PCD, Raw PCD) improve per-grasp reward but fail to achieve full pile clearing (Table **??**). BC→RL initializes from the trained BC policy and reaches the highest episode reward while maintaining near-complete clearing.

step per cycle) across all parallel environments and performs one PPO update; total grasp experience after k iterations is therefore $4kN_{\text{env}}$. The outcome components (g, α, s) from Section **??** define the scalar reward:

$$r = g \cdot \alpha \cdot s, \tag{14}$$

for successful grasps ($g > 0$), and $r = \rho_{\text{fail}} = -1$ for failed grasps. This multiplicative form means that a high reward requires all three components to be large: many logs grasped, well aligned, and with a level grapple. The reward is entirely per-grasp; there is no episode-level completion bonus. Global pile-clearing ability thus emerges from the policy's targeting strategy rather than from explicit incentives to clear.

D. BC-Initialized RL Fine-Tuning

To combine BC's full-trajectory coverage with RL's ability to optimize per-grasp metrics, we initialize a PPO agent from the trained BC policy and fine-tune with on-policy experience [**?**], [**?**].

The BC actor weights (PointNet encoder and actor MLP) are loaded into the PPO actor; the critic is initialized randomly and the optimizer starts fresh. We use an *asymmetric actor-critic* [**?**]: the actor observes the raw point cloud (same as BC), while the critic is a trainable MLP that receives the privileged 128D pose state vector (top-K log poses). This asymmetry allows the critic to learn a more accurate value function from compact state information, while the actor retains the vision-based input it will use at deployment.

We freeze the BC-trained PointNet encoder during RL fine-tuning; fine-tuning the full network—even with a $5\times$ reduced encoder learning rate—causes feature drift and training collapse within $\sim$100 iterations. This diverges from DAPG [**?**], which fine-tunes all weights; the frozen encoder is necessary because the asymmetric critic cannot propagate gradients through the encoder, so an unfrozen encoder drifts without constraint.

Conservative hyperparameters prevent the RL updates from overwriting the BC-learned policy. We set the initial policy noise $\sigma_0 = 0.05$ (vs. 1.0 for from-scratch RL), reducing exploration to small perturbations around the BC mean; values up to $\sigma_0 = 0.1$ produce comparable results, while $\sigma_0 = 0.3$ causes training collapse. The learning rate is reduced to 10^{-4} (vs. 3×10^{-4}), the clipping ratio tightened to $\epsilon = 0.1$ (vs. 0.2), and the entropy coefficient set to zero. We collect 8 steps per environment per iteration (vs. 4 for from-scratch RL) for smoother updates.

VI. Experiments

A. Setup

All experiments use the same crane model, rack geometry, and FSM controller, and run on a single NVIDIA RTX 4070 Laptop GPU (8 GB VRAM). Training uses 32 parallel environments for pose-based policies and 20 for point cloud policies. Piles contain 200 cylindrical logs ($\sim$0.3 m diameter, $\sim$3 m length) spawned in layered grid patterns. The log count is fixed across all experiments so that clearing metrics are directly comparable between methods; domain randomization varies only the pile geometry (spawn pattern, row jitter, and layer offsets).

Each episode runs for a fixed budget of $H = 30$ grasp attempts (see Section III-A for the horizon rationale). An episode terminates early if no logs remain on the rack (whether grasped or knocked out of bounds), though only intentionally grasped logs count toward the clearing rate. For evaluation, we run 100 episodes across 32 parallel environments on identical random seeds to ensure each method faces the same pile configurations. All metrics are reported as per-episode mean $\pm$ standard deviation. Training reward curves (Fig. **??**) show single training runs; the reported variability comes from averaging over episodes within a run rather than across independent seeds.

B. Metrics

We report per-grasp and per-episode metrics, all as per-episode mean $\pm$ standard deviation unless noted:

- Grasp success rate: fraction of grasps with $g > 0$.
- Throughput: mean logs grasped per successful grasp.
- Alignment: grasp-count-weighted mean $\alpha \in [0, 1]$ per episode (grasps capturing more logs contribute proportionally more).
- Stability: mean $s \in [0, 1]$ over successful grasps per episode.
- Pile clearing rate: fraction of initial logs intentionally grasped and removed by episode end (knocked-off logs are excluded).

TODO: Insert screenshot of multiple parallel environments, each showing a crane above a rack with 200 logs in different randomized pile configurations (varying spawn patterns, row jitter, layer offsets). Should convey the domain randomization and parallelized training setup.

Fig. 3: The grasp-and-remove crane environment. Each parallel environment contains a log loader crane positioned above a rack with 200 cylindrical logs spawned in randomized configurations. At each step, the policy selects a grasp target and a fixed FSM controller executes the pick sequence.

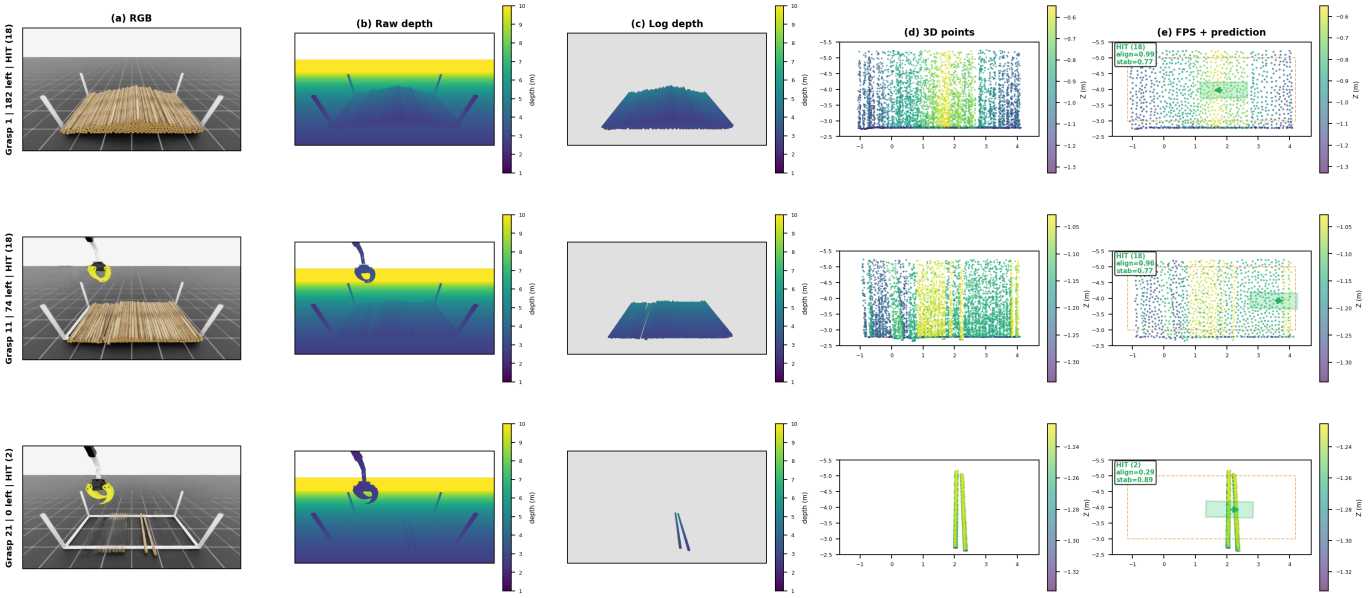

Fig. 4: Observation pipeline at three episode stages. Each row: (a) RGB, (b) depth, (c) segmented depth, (d) 3D back-projection, (e) FPS point cloud with predicted grasp target (green = hit, red = miss). Top: dense pile; middle: partially cleared; bottom: sparse endgame.

TODO: Insert clearing progression curves: fraction of pile cleared (mean ± shaded std across episodes) vs. grasp attempt number, one curve per method (Heuristic, RL, BC, BC+RL). This is the paper's core argument figure.

Fig. 5: Clearing progression: fraction of 200-log pile cleared vs. grasp attempt number (mean ± std over 100 episodes). RL plateaus at ∼60% while BC, BC+RL, and the heuristic all approach full clearing.

- Knocked-off percentage: fraction of initial logs knocked outside workspace bounds.

Our primary visualization is the clearing progression curve: for each method, we plot fraction of pile cleared (mean ± std across episodes) as a function of grasp attempt number (Fig. **??**). This curve captures the full clearing dynamics, including where methods diverge and where RL plateaus.

VII. Results

A. Overall Performance

Table **??** summarizes the evaluation metrics. All four methods share the same controller, pile configuration, and evaluation protocol; the only differences are the targeting policy and its observation. From-scratch RL uses a symmetric actor-critic in which both actor and critic process point clouds through independent PointNet encoders. BC+RL uses an asymmetric actor-critic [**?**] in which the actor retains the frozen BC-trained PointNet encoder while the critic is a trainable MLP that receives the privileged state vector (Section **??**).

The heuristic expert clears $97.9 \pm 4.4\%$ of the pile with a grasp success rate of 88.4%. Both BC ($98.6 \pm 2.2\%$) and BC+RL ($98.3 \pm 1.6\%$) achieve comparable clearing from raw point clouds—notably, without requiring semantic segmentation. BC+RL achieves the best grasp success rate (87.0%) and grapple stability (0.950), demonstrating that RL fine-tuning improves per-grasp execution quality. From-scratch RL stalls at roughly 60% clearing regardless of observation type (Table **??**).

Qualitatively, we observe three regimes (Fig. **??**): (i) early clearing, where dense piles yield high-throughput grasps for all methods; (ii) mid clearing, where RL diverges from the other methods; and (iii) endgame clearing, where remaining logs form a thin sparse layer and BC+RL's improved success rate becomes most valuable.

B. BC+RL Performance

BC+RL fine-tuning improves per-grasp metrics while preserving BC's full-trajectory clearing ability. The most strik-

TABLE III: Evaluation results across 100 episodes with randomized 200-log pile configurations (mean ± std). All learned methods use raw (unsegmented) point cloud observations processed by a PointNet encoder. RL uses a symmetric actor-critic (shared encoder); BC+RL uses an asymmetric actor-critic (frozen BC encoder for actor, state-based MLP critic).

Method	Obs.	Throughput	Align. α	Stab. s	Grasp succ. (%)	Clear rate (%)	Knocked off (%)
Heuristic	Pose	9.40 ± 1.17	0.914 ± 0.027	0.789 ± 0.024	88.4 ± 10.1	97.9 ± 4.4	1.4 ± 1.6
RL	Raw PCD	**TODO:** Fill after RL (Raw PCD, sym) evaluation					
BC	Raw PCD	9.23 ± 0.95	0.905 ± 0.033	0.824 ± 0.025	82.5 ± 10.3	98.6 ± 2.2	**TODO:** TBD
BC→RL	Raw PCD	8.66 ± 0.92	0.908 ± 0.028	$\mathbf{0.950 \pm 0.016}$	$\mathbf{87.0 \pm 9.3}$	98.3 ± 1.6	1.2 ± 1.0

ing improvement is in grapple stability: 0.950 ± 0.016 vs. 0.824 ± 0.025 for BC and 0.789 ± 0.024 for the heuristic. This indicates that RL optimizes the grapple's levelness during lifting, likely by refining the vertical component of the grasp target. Grasp success rate also improves from BC's 82.5% to 87.0%, reducing wasted grasps—particularly in the endgame where each miss consumes budget. Throughput is slightly lower (8.66 vs. 9.23 for BC), suggesting that BC+RL favors more conservative, reliable grasps over maximum-throughput grasps. The mean episode reward of 166.4 is the highest across all methods, confirming that BC+RL simultaneously improves all reward components.

C. BC Performance

BC achieves $98.6 \pm 2.2\%$ clearing from raw point clouds, exceeding the heuristic expert's $97.9 \pm 4.4\%$ (though the difference is within standard deviations). Per-grasp quality metrics are close to the heuristic: throughput 9.23 ± 0.95 (vs. 9.40), alignment 0.905 ± 0.033 (vs. 0.914). Grasp success rate ($82.5 \pm 10.3\%$) is lower than the heuristic's 88.4%, with the gap concentrated in the endgame where sparse remaining logs are harder to localize from depth alone. However, BC compensates by maintaining effective targeting throughout the clearing trajectory, ultimately achieving comparable clearing despite more missed grasps.

Throughput, alignment, and stability are computed only over successful grasps ($g > 0$). The near-match in these conditional metrics indicates that the BC policy has captured the expert's targeting and alignment strategy: when it connects, it grasps just as effectively as the privileged-state expert.

D. RL Performance

From-scratch RL achieves competitive per-grasp reward during training (Fig. **??**): RL (Pose) converges to a mean episode reward of ∼95–97 within 80 iterations, while RL (PCD) reaches ∼85–90 after 800+ iterations. However, high training reward does not translate to pile-clearing performance.

To test whether observation type affects performance, we trained three RL variants with a symmetric actor-critic: privileged pose observations (128D), segmented PCD, and raw PCD. All achieve roughly similar clearing (Table **??**), confirming that RL's failure is fundamental to the exploration landscape rather than a limitation of the observation representation. In contrast, observation type does matter for BC: raw PCD (98.6% clearing, 82.5% success) outperforms segmented PCD (96.3% clearing, 72.0% success), showing that segmentation provides no benefit and slightly hurts performance (Table **??**).

TABLE IV: Observation type ablation for RL and BC. All RL variants use symmetric actor-critic. BC variants use PointNet with identical architecture. Clearing rate is mean ± std over 100 episodes. RL fails regardless of observation type; for BC, raw PCD outperforms segmented PCD.

Method	Observation	Clear rate (%)
RL	Pose (128D)	**TODO:** Eval needed
RL	Seg. PCD	**TODO:** Eval needed (training: ∼58.6%)
RL	Raw PCD	**TODO:** Eval needed (training: ∼56%)
BC	Seg. PCD	96.3 ± 5.0
BC	Raw PCD	98.6 ± 2.2

The RL policies converge to narrow targeting patterns that yield adequate per-grasp reward in dense configurations but fail to adapt as the pile thins. **TODO:** Update with eval numbers for best RL variant: clearing %, success rate, throughput, alignment, stability.

E. Per-Grasp Analysis by Remaining Logs

Figure **??** shows per-grasp throughput as a function of remaining logs in the pile.

VIII. Discussion and Limitations

Why RL fails from scratch. From-scratch RL achieves competitive per-grasp reward but stalls at roughly 60% clearing regardless of observation type (Table **??**). Because the per-grasp reward provides no direct incentive for global clearing, the policy converges to a narrow targeting pattern that yields adequate reward in dense configurations but does not adapt as the pile geometry changes through clearing. Once the easily accessible logs are exhausted, the fixed strategy produces missed grasps that consume the episode budget. This failure is not a perception limitation—RL with privileged pose observations fares no better—but rather a consequence of the exploration landscape: the policy never discovers the full diversity of pile states encountered during a complete clearing trajectory.

Why BC+RL succeeds. BC initialization provides exactly the coverage that from-scratch RL lacks: the demonstration data spans the full clearing trajectory from dense stacks to sparse endgames [**?**]. RL fine-tuning then optimizes per-grasp

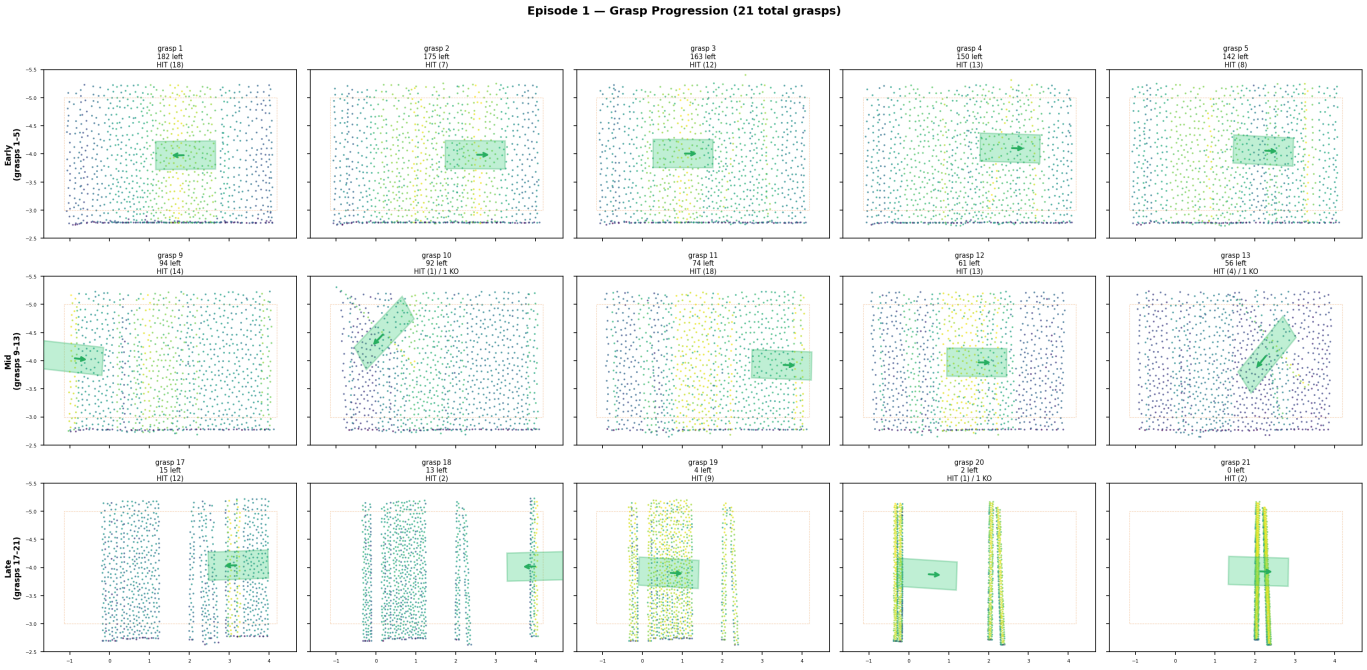

Fig. 6: Grasp progression for a single BC (PCD) episode clearing 200 logs in 21 grasps. Each panel shows the FPS-downsampled point cloud with the predicted grasp target (green rectangle). Top row: early grasps into the dense pile; middle row: mid-episode with thinning pile; bottom row: endgame grasps targeting sparse remaining logs. The pile transitions from a dense ordered stack to a thin irregular layer.

TODO: Insert per-remaining-logs analysis: mean logs grasped per attempt (y-axis) vs. number of remaining logs (x-axis), one curve per method (Heuristic, RL, BC, BC+RL).

Fig. 7: Per-grasp throughput as a function of remaining logs. From-scratch RL throughput degrades as the pile thins, while the heuristic, BC, and BC+RL maintain higher throughput across pile states.

execution quality without needing to rediscover the global targeting strategy. The dramatic stability improvement (0.950 vs. 0.824) suggests that RL learns to refine the vertical positioning of grasps, producing more level lifts. Freezing the BC-trained encoder is critical: the asymmetric critic receives only the state vector and cannot propagate gradients through the encoder, so an unfrozen encoder drifts without constraint. The conservative hyperparameters ($\sigma_0 \leq 0.1$, reduced learning rate, tight clipping) further prevent the RL updates from overwriting the BC-learned policy.

Raw PCD sufficiency. Raw (unsegmented) point clouds achieve 98.6% clearing via BC and 98.3% via BC+RL, matching or exceeding the segmented-PCD BC baseline (96.3% in our earlier experiments with segmented input). This has practical implications for sim-to-real transfer: removing the dependency on a trained segmentation model simplifies the perception pipeline and eliminates a potential failure mode. The PointNet encoder learns to extract task-relevant geometry directly from the raw depth signal.

BC+RL trade-offs. BC+RL's throughput (8.66 logs/grasp) is lower than BC's (9.23), suggesting a reliability–throughput trade-off: the fine-tuned policy favors more conservative grasps that succeed more often (87.0% vs. 82.5%) and produce more stable lifts (0.950 vs. 0.824), at the cost of slightly fewer logs per grasp. Whether this trade-off is desirable depends on the application; in practice, the higher success rate and lower variance of BC+RL may be preferred for autonomous operation.

Limitations. Our grasp-and-remove design isolates targeting and enables fast experimentation, but has limitations. First, grasp success is approximated by proximity after lift, rather than explicit contact constraints. Second, removed logs are teleported rather than placed, so the task omits deposition planning and re-contact dynamics. Third, the PointNet encoder operates on unordered point sets without explicit spatial structure; architectures that exploit local geometry (e.g., PointNet++ [**?**], point transformers) may improve endgame targeting. Fourth, all experiments are in simulation; sim-to-real transfer with domain randomization is a natural next step. Despite these simplifications, the environment captures the essential challenges of sequential multi-log grasping from dense piles.

IX. Conclusion

We presented a grasp-and-remove crane environment for studying grasp target selection in dense log-pile clearing at scale, and compared four methods—heuristic, from-scratch RL, BC, and BC+RL—on a shared controller and evaluation

protocol. Our results show that from-scratch RL fails to clear dense piles regardless of observation type, stalling at roughly 60%. BC trained on raw (unsegmented) point clouds achieves 98.6% clearing, matching the 97.9% privileged-state heuristic. BC+RL fine-tuning achieves the strongest overall performance: 87.0% grasp success, 0.950 stability, and 98.3% clearing, demonstrating that demonstration data provides the exploration curriculum that RL cannot discover on its own. The success of raw point clouds—without semantic segmentation—simplifies the perception pipeline for future sim-to-real transfer. Future work will explore sim-to-real deployment with domain randomization, richer point cloud architectures (e.g., PointNet++, point transformers), and curriculum learning over pile complexity.

ACKNOWLEDGMENT

[Redacted for anonymity]